

COURSE REFERENCE

Java Reference

Foundations to Expert · Java 25 LTS

Generated July 2026 · anchored to the Java Language Specification & API, SE 25

How to use. This is a lookup reference, not a lesson. Browse by topic area, or find a term fast in the **alphabetical index** at the back. Each entry leads with a one-sentence essence; the detail beneath gives the mechanism, form, and a worked example.

■ CONCEPT ■ SYNTAX

Contents

Language & Runtime · 7	3
Types & Variables · 7	5
Strings & Text · 8	9
Operators & Conversion · 5	13
Console I/O · 4	16
Dates & Times · 8	18

TOPIC AREA

7 entries

Language & Runtime

AT A GLANCE

Java source is compiled once into platform-neutral bytecode, then run on any machine by the JVM. Two tools bracket the process: `javac` compiles, `java` launches. Knowing which tool does which step — and that the JVM, not the compiler, produces the final machine code — is the foundation everything else sits on.

How Java runs CONCEPT

Java source is compiled to platform-neutral bytecode, which the JVM then executes at run time — so the same compiled program runs on any machine with a JVM.

MECHANISM

`javac` turns `Foo.java` into `Foo.class` (bytecode, not machine code). `java` starts the JVM, which loads that bytecode and — via JIT compilation and interpretation — turns it into native machine code as the program runs.

WHY

The bytecode layer is the portability seam: compile once, and any operating system with a JVM can run the result. That is the whole 'write once, run anywhere' claim, made concrete.

EXAMPLE

```
javac Hello.java    # source -> Hello.class (bytecode)
java Hello          # JVM loads bytecode -> runs it
```

SOURCE [JLS SE 25](https://docs.oracle.com/javase/specs/jls/se25/html/index.html) <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO [javac](#) · [java](#) · [bytecode](#) · [JDK vs JRE vs JVM](#)

javac SYNTAX

The compiler. It reads a `.java` source file and produces a `.class` bytecode file.

FORM

`javac` takes the full file name, extension included.

EXAMPLE

```
javac Hello.java    # requires the .java extension
# produces: Hello.class
```

SOURCE [JLS SE 25](https://docs.oracle.com/javase/specs/jls/se25/html/index.html) <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO [java](#) · [How Java runs](#) · [bytecode](#)

java SYNTAX

The launcher. It starts the JVM on a compiled class and runs its main method.

FORM

java takes the class NAME only — no .java, no .class extension.

MECHANISM

The final step from bytecode to native machine code happens HERE, inside the JVM — not in javac. javac stops at bytecode.

EXAMPLE

```
java Hello    # bare class name, NO extension
# NOT: java Hello.class  (wrong)
```

SOURCE [JLS SE 25](https://docs.oracle.com/javase/specs/jls/se25/html/index.html) <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO [javac](#) · [How Java runs](#) · [main method](#)

JDK vs JRE vs JVM CONCEPT

Three nested layers: the JDK builds programs, the JRE runs them, and the JVM inside it actually executes the bytecode.

MECHANISM

JDK $\supset$ JRE $\supset$ JVM. The JDK adds developer tools (javac and friends) on top of the JRE. The JRE bundles the JVM plus the standard libraries. The JVM is the engine that executes bytecode.

USE WHEN

You need the JDK to write and compile Java. A machine that only runs finished Java programs needs just the runtime.

SOURCE [JLS SE 25](https://docs.oracle.com/javase/specs/jls/se25/html/index.html) <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO [How Java runs](#) · [javac](#) · [java](#)

bytecode CONCEPT

Platform-neutral instructions stored in .class files — the JVM's 'machine code', not the CPU's.

MECHANISM

javac emits bytecode: a compact instruction set the JVM understands. It is not tied to any real processor; the JVM translates it to actual CPU instructions at run time.

WHY

Because bytecode is CPU-independent, one .class file runs unchanged on any JVM — the mechanism behind Java's portability.

SOURCE [JLS SE 25](https://docs.oracle.com/javase/specs/jls/se25/html/index.html) <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO [How Java runs](#) · [javac](#) · [JDK vs JRE vs JVM](#)

main method SYNTAX

The program's entry point — where the JVM begins execution.

FORM

```
public static void main(String[] args) {  
    System.out.println("Hello");  
}
```

MECHANISM

public (callable from outside), static (no object needed to start), void (returns nothing), args holds any command-line arguments.

NOTE

Java 25 previews a simplified entry point (instance main methods / simple source files), but the classic signature above is the stable, universal form.

SOURCE [JLS SE 25](https://docs.oracle.com/javase/specs/jls/se25/html/index.html) <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO [java · How Java runs](#)

Representative — confirm against your actual lesson.

import SYNTAX

Tells the compiler which package a class name refers to -- required for every class except those in `java.lang`, which loads automatically.

MECHANISM

Java's standard library is organized into packages (folders of related classes). `java.lang` (`String`, `Math`, etc.) is auto-loaded; every other package's classes must be explicitly imported by their fully-qualified path before use.

FORM

One import per line, above and outside every class declaration -- never inside a class body or method.

EXAMPLE

```
import java.util.Scanner;  
  
public class Foo {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
    }  
}
```

COMMON ERROR

Placing import inside the class body gives "illegal start of type" -- the compiler is already past the import-reading phase and expects a class member instead. Omitting it entirely for a non-`java.lang` class gives "cannot find symbol."

SOURCE [JLS SE 25 SS7.5](https://docs.oracle.com/javase/specs/jls/se25/html/jls-7.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-7.html>

SEE ALSO [How Java runs · Scanner](#)

TOPIC AREA

7 entries

Types & Variables

AT A GLANCE

Java values are one of two kinds: primitives (raw numbers, characters, booleans, stored by value) or references (objects, addressed by reference). Primitive sizes are fixed and identical on every machine. The distinction between the two kinds is what makes `==` behave the way it does.

primitive types CONCEPT

The eight built-in value types, with fixed sizes identical on every machine, stored directly (not as objects).

THE EIGHT PRIMITIVES

TYPE	BITS	CATEGORY	NOTES
byte	8	signed integer	two's complement
short	16	signed integer	two's complement
int	32	signed integer	default integer type
long	64	signed integer	literal suffix <code>L</code>
float	32	floating-point	IEEE 754, suffix <code>f</code>
double	64	floating-point	IEEE 754, default decimal
char	16	unsigned integer	UTF-16 code unit
boolean	—	boolean	true / false only

MECHANISM

A primitive variable holds the value itself, not a reference to an object elsewhere. Copying a primitive copies the value.

SOURCE [JLS §4.2](https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html>

SEE ALSO `int` · `double` · `variable declaration` · `==` vs `.equals()`

Representative — confirm against your actual lesson.

int SYNTAX

A 32-bit signed whole number — Java's default type for integer values.

RANGE

−2,147,483,648 to 2,147,483,647 (-2^{31} to $2^{31}-1$), stored as 4-byte two's complement.

USE WHEN

Counting, indexing, and whole-number math within range. Switch to long if the value might exceed the range.

EXAMPLE

```
int count = 42;
int sum = a + b;
```

SOURCE [JLS §4.2](https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html>

SEE ALSO [primitive types · double](#)

Representative — confirm against your actual lesson.

double SYNTAX

A 64-bit IEEE 754 floating-point number — Java's default type for decimals.

USE WHEN

Any calculation with fractional values where tiny rounding error is acceptable — physics, statistics, graphics.

EXAMPLE

```
double g = 9.81;
double avg = total / (double) n;
```

SOURCE [JLS §4.2](https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html>

SEE ALSO [primitive types · int](#)

Representative — confirm against your actual lesson.

variable declaration SYNTAX

Name a typed storage slot and optionally give it a value: type name = value;

FORM

```
int count = 0;
String name = "Roque";
```

MECHANISM

Declares a named, typed storage slot. A primitive variable holds its value directly; a reference variable holds a pointer to an object elsewhere in memory.

SOURCE [JLS §4](https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html>

SEE ALSO [primitive types · int · String · var](#)

var SYNTAX

Tells the compiler to infer a local variable's type from its initializer, then lock it in exactly as if the type had been written out by hand -- not dynamic typing.

MECHANISM

The compiler resolves the initializer's type at compile time and rewrites the declaration internally as if that type had been typed explicitly. `var` itself leaves zero trace in the compiled bytecode.

LEGAL SCOPE

Only a local variable declaration, inside a method/constructor/block, WITH an initializer on the same line. Illegal for fields, method parameters, and return types -- none of those have an immediate initializer for the compiler to infer from at that exact point.

EXAMPLE

```
var count = 10;      // inferred as int
count = "ten";      // fails -- count IS an int, same as always
```

THE PYTHON TRAP

`var` looks flexible but is not: once inferred, the variable is exactly as statically typed as if you had written `int/String/` etc. by hand.

SOURCE [JLS SE 25 SS14.4](https://docs.oracle.com/javase/specs/jls/se25/html/jls-14.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-14.html>

SEE ALSO [variable declaration](#) · [primitive types](#)

literal defaults & suffixes CONCEPT

A bare whole-number literal defaults to `int`; a bare decimal literal defaults to `double` -- and this default is decided before the compiler ever looks at the target variable's type.

MECHANISM

3000000000 (no suffix) is invalid for a long variable not because long is too small, but because the literal itself is already too big to exist as the `int` it defaults to. The `L` or `f` suffix tells the compiler the literal's type from the very first moment, skipping the default entirely.

FORM

SUFFIX	FORCES LITERAL TYPE	EXAMPLE
(none, whole number)	<code>int</code>	100
L	<code>long</code>	3000000000L
(none, decimal)	<code>double</code>	3.14
f	<code>float</code>	3.14f

NOTE

`double` -> `float` is narrowing on precision, not range -- 3.14 fits in a `float`'s range but loses precision, so it needs the `f` suffix or an explicit cast even though the value looks small.

SOURCE [JLS SE 25 SS3.10](https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html>

SEE ALSO [primitive types](#) · [widening & narrowing conversion](#)

constant expression narrowing CONCEPT

A narrowing conversion is allowed **WITHOUT** a cast only when three conditions all hold: it's an assignment context, the source is a compile-time constant, and the value fits the target type's range.

THE THREE GATES (ALL REQUIRED)

- Gate 1** Assignment context (a declaration or simple =) -- NEVER a method-call argument, even with an in-range literal.
- Gate 2** Source is a compile-time constant -- a literal, or a final variable initialized with a constant expression. A plain (non-final) variable never qualifies, regardless of how 'obviously stable' its value looks.
- Gate 3** The value is representable in the target type's range.

VERIFIED BEHAVIOR (JDK 21)

CODE	COMPILES?
byte b = 100;	yes -- literal, in range
int x = 100; byte b = x;	no -- x is not final
final int x = 100; byte b = x;	yes -- final + constant + in range
final int x = 200; byte b = x;	no -- out of range, final doesn't help
void f(byte b); f(100);	no -- method call, not assignment context

WHY

The compiler never traces whether a plain variable's value actually changes before use -- that would require arbitrarily deep program analysis. `final` is a simple, syntactic, always-checkable substitute: it lets the compiler treat the variable exactly like a literal.

SOURCE **JLS SE 25 SS5.2** <https://docs.oracle.com/javase/specs/jls/se25/html/jls-5.html>

SEE ALSO widening & narrowing conversion · var

TOPIC AREA

8 entries

Strings & Text

AT A GLANCE

A `String` is an immutable object — every method that seems to change it actually returns a new one. That single fact explains the literal pool, why `==` is the classic trap for anyone coming from Python, and why `StringBuilder` exists. Text blocks add readable multi-line literals, with precise, easy-to-miss whitespace rules.

String CONCEPT

An immutable sequence of characters (UTF-16) — a reference type, not a primitive, with special literal and + syntax.

MECHANISM

A String is an object holding a fixed character sequence. The language gives it two conveniences no other class gets: double-quote literals ("hi") and the + concatenation operator.

EXAMPLE

```
String s = "hello";
String greeting = s + ", world";
```

SOURCE [java.lang.String \(API\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html>

SEE ALSO [String immutability](#) · [== vs .equals\(\)](#) · [StringBuilder](#) · [string pool](#)

String immutability CONCEPT

Once a String exists, its contents can never change — every 'modifying' method returns a brand-new String and leaves the original untouched.

MECHANISM

Methods like `toUpperCase()`, `trim()`, and `replace()` do not edit the String in place. They compute and return a new String object; the original is unchanged.

PROOF

Verified empirically in-lesson: calling `s.toUpperCase()` and then re-checking `s` showed `s` was untouched — the uppercased text only exists in the returned value.

WHY IT MATTERS

Immutability is what makes the literal pool and thread-safety possible — a shared String can never be altered out from under another user of it.

EXAMPLE

```
String s = "hi";
s.toUpperCase();           // result thrown away!
String big = s.toUpperCase(); // correct: capture it
```

SOURCE [java.lang.String \(API\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html>

SEE ALSO [string pool](#) · [StringBuilder](#) · [String](#) · [date-time immutability](#)

string pool CONCEPT

Identical String literals share one single object in a cache called the pool — safe only because Strings can never change.

MECHANISM

When the same literal text appears more than once, Java reuses one pooled object rather than making duplicates. Because Strings are immutable, sharing is safe — no user of the shared object can mutate it.

CONSEQUENCE

Two identical literals are literally the same object, so `==` between them happens to return `true`. This is an implementation detail — never rely on it for comparison.

SOURCE [JLS §3.10.5](https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html>

SEE ALSO [String immutability](#) · [new String\(\)](#) · `==` vs `.equals()`

new String() SYNTAX

Explicitly constructs a fresh String object that deliberately does NOT come from the pool — even if an identical literal already exists.

MECHANISM

`new String("a")` forces a distinct object. So `new String("a") == "a"` is `false` (different objects), even though their contents are equal.

USE WHEN

Almost never in normal code. Its main value is illustrating the pool: it is the deliberate way to opt out of literal sharing.

EXAMPLE

```
String a = "a";
String b = new String("a");
a == b;           // false – different objects
a.equals(b);     // true  – same contents
```

SOURCE [java.lang.String \(API\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html>

SEE ALSO [string pool](#) · `==` vs `.equals()` · [String immutability](#)

== vs .equals() CONCEPT

`==` asks 'are these the same object?' (reference identity); `.equals()` asks 'do these have the same contents?' — for Strings, always use `.equals()`.

MECHANISM

`==` compares references: are both variables pointing at the exact same object? `.equals()` (as `String` overrides it) compares the actual character sequences.

THE PYTHON TRAP

In Python, `==` compares values. In Java, `==` on any object compares identity, not value. Carrying the Python habit over is the classic source of `String` bugs.

EXAMPLE

```
String a = new String("hi");
String b = new String("hi");
a == b;           // false – two objects
a.equals(b);      // true  – same text
```

SOURCE [JLS §15.21](https://docs.oracle.com/javase/specs/jls/se25/html/jls-15.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-15.html>

SEE ALSO [string pool](#) · [new String\(\)](#) · [String immutability](#)

StringBuilder CONCEPT

A mutable character buffer — the right tool for building a `String` piece by piece, because it edits one buffer in place instead of creating a new object per step.

MECHANISM

`StringBuilder` holds a growable internal buffer (initial capacity 16). `append()` and `insert()` modify that one buffer; you call `toString()` at the end to get the finished `String`.

USE WHEN

Assembling text in a loop, or from many pieces. For a couple of + operations, plain `String` is fine and clearer.

EXAMPLE

```
StringBuilder sb = new StringBuilder();
for (String w : words) sb.append(w).append(' ');
String line = sb.toString();
```

SOURCE [java.lang.StringBuilder \(API\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/StringBuilder.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/StringBuilder.html>

SEE ALSO [String immutability](#) · [String](#)

text blocks SYNTAX

Multi-line String literals delimited by triple quotes — readable JSON, HTML, or SQL without escaped newlines.

FORM

Open with `"""` followed immediately by a newline; write the content on following lines; close with `"""`. The result is an ordinary String.

EXAMPLE

```
String json = """
    {
        "name": "Roque"
    }
    """;
```

SOURCE [JLS §3.10.6](https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html>

SEE ALSO [text block whitespace](#) · [String](#)

text block whitespace SYNTAX

Text blocks strip the whitespace you didn't mean and keep the whitespace you did — via a few precise rules worth memorizing.

INCIDENTAL LEADING WHITESPACE

The common indentation shared by all lines (measured against the least-indented line and the closing `"""` position) is stripped automatically, so you can indent the block to match your code without that indentation appearing in the string.

TRAILING WHITESPACE

Whitespace at the end of each line is removed automatically.

KEEP A TRAILING SPACE: `\s`

End a line with the `\s` escape to preserve exactly one trailing space that would otherwise be stripped.

JOIN LINES: TRAILING `\`

End a line with a backslash to suppress its newline — the next line continues the same logical line.

SOURCE [JLS §3.10.6](https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html>

SEE ALSO [text blocks](#) · [String](#)

TOPIC AREA

5 entries

Operators & Conversion

AT A GLANCE

Operator precedence decides evaluation order; binary numeric promotion decides whether an arithmetic expression computes as whole numbers or decimals. Casting moves a value between differently-sized boxes -- automatic when widening, explicit and truncating (never rounding) when narrowing. The Math class wraps common numeric operations with its own, sometimes surprising, return-type rules.

operator precedence CONCEPT

Multiplication, division, and modulus evaluate before addition and subtraction, left-to-right within the same tier -- not the order the expression is read left-to-right overall.

MECHANISM

* / % share the top arithmetic tier; + - sit below. Same-tier operators evaluate left-to-right. Precedence changes WHICH VALUE results, not whether the code compiles.

EXAMPLE

```
2 + 3 * 4      // 14, not 20 -- * runs first
10 - 4 / 2 + 3 % 2  // 9: (4/2)=2, (3%2)=1, then 10-2+1
```

SOURCE [JLS SE 25 SS15](https://docs.oracle.com/javase/specs/jls/se25/html/jls-15.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-15.html>

SEE ALSO [integer division & modulus](#) · [binary numeric promotion](#)

integer division & modulus CONCEPT

int / int truncates any fractional result; % returns the remainder and takes the SAME SIGN as the dividend (left operand) -- a real trap coming from Python, where % takes the divisor's sign.

MECHANISM

9 / 4 is pure integer arithmetic: truncates to 2, discarding the remainder entirely. 9 % 4 gives that discarded remainder: 1.

THE PYTHON TRAP

-15 % 4 is -3 in Java (sign follows the dividend, -15), but 1 in Python (sign follows the divisor).

EXAMPLE

```
9 / 4      // 2 (truncated)
9 % 4      // 1 (remainder)
-15 % 4    // -3 (NOT 1 -- Java differs from Python here)
```

SOURCE [JLS SE 25 SS15.17](https://docs.oracle.com/javase/specs/jls/se25/html/jls-15.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-15.html>

SEE ALSO [binary numeric promotion](#) · [operator precedence](#)

binary numeric promotion CONCEPT

If either operand of an arithmetic operator is a floating-point type, the other operand is promoted to match, and the WHOLE operation computes in floating-point.

MECHANISM

The / operator itself is neutral -- it isn't 'integer division' or 'decimal division' by nature. It executes whatever arithmetic the operand types dictate. `7 / 2` is pure int arithmetic (both operands int) and truncates to 3. `7.0 / 2` promotes the 2 to 2.0 first, so the whole division is floating-point: 3.5.

EXAMPLE

```
7 / 2      // 3    (both int)
7.0 / 2    // 3.5  (2 promoted to double)
```

SOURCE [JLS SE 25 SS5.6.2](https://docs.oracle.com/javase/specs/jls/se25/html/jls-5.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-5.html>

SEE ALSO [integer division & modulus](#) · [widening & narrowing conversion](#)

widening & narrowing conversion CONCEPT

Widening (small box to big box) is automatic and always safe. Narrowing (big box to small box) requires an explicit cast, and for decimal-to-integer casts, truncates toward zero -- it never rounds.

MECHANISM

`double y = someInt;` needs nothing extra. `int x = (int) someDouble;` requires the explicit (int) because information could be lost, and Java insists you say so in the code.

TRUNCATION DIRECTION

Casting a decimal to an integer type chops everything after the decimal point, always toward zero -- never toward negative infinity. (int) -9.99 is -9 (closer to zero), NOT -10.

EXAMPLE

```
int x = 10;
double y = x;           // widening, automatic

double d = 9.99;
int z = (int) d;        // narrowing, explicit -- z is 9, not 10
int w = (int) -9.99;    // w is -9, not -10
```

SOURCE [JLS SE 25 SS5](https://docs.oracle.com/javase/specs/jls/se25/html/jls-5.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-5.html>

SEE ALSO [constant expression narrowing](#) · [literal defaults & suffixes](#)

Math class SYNTAX

A utility class of static numeric methods -- each has a specific, sometimes-surprising return type that does not always match intuition.

RETURN TYPES (VERIFIED AGAINST JDK 21/25)

METHOD	RETURNS	NOTE
Math.pow(a, b)	double	ALWAYS double, even for whole-number results: pow(2,10) prints 1024.0
Math.abs(x)	same as input	Overloaded per type (abs(int)->int, abs(double)->double) -- preserves, doesn't convert
Math.round(double)	long	floor(x + 0.5). round(-4.5) is -4, not -5 -- rounds toward positive infinity on ties
Math.round(float)	int	Different overload, different return type than the double version
Math.sqrt(x)	double	Always double regardless of input type
Math.min / Math.max	promoted match	Mixed int/double args promote the int to double first, same as binary numeric promotion
Math.random()	double	Range [0.0, 1.0) -- inclusive of 0, exclusive of 1, never exactly 1.0

SOURCE [java.lang.Math \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Math.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Math.html>

SEE ALSO [binary numeric promotion](#) · [widening & narrowing conversion](#)

TOPIC AREA

4 entries

Console I/O

AT A GLANCE

System.out offers three output methods that differ only in line-ending and templating behavior; printf's placeholders match arguments strictly by position, and a type mismatch is always a runtime failure, never a compile error. Scanner reads input by tokenizing on whitespace -- the source of its single most common bug, where a leftover newline from one read silently empties the very next `nextLine()` call.

System.out.print / println / printf SYNTAX

print outputs exactly what you give it; println does the same and appends one line terminator; printf builds output from a format-string template with placeholders.

MECHANISM

System.out is a `PrintStream` instance. print/println differ only in the trailing line terminator. printf takes a format string plus a list of values and substitutes them into %-placeholders.

EXAMPLE

```
System.out.print("Hi");    // no newline after
System.out.println("Hi"); // newline after
System.out.printf("Age: %d%n", 25); // "Age: 25"
```

SOURCE [java.io.PrintStream \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/io/PrintStream.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/io/PrintStream.html>

SEE ALSO [printf format specifiers](#) · [Scanner](#)

printf format specifiers SYNTAX

Placeholders are matched to arguments strictly by position, left to right -- and a type mismatch between a placeholder and its argument compiles fine but throws at runtime.

COMMON SPECIFIERS

SPECIFIER	EXPECTS	NOTE
%d	integer types	Strict -- a non-integer argument throws <code>IllegalFormatConversionException</code>
%s	anything	Flexible -- calls <code>.toString()</code> on any argument, never fails on type
%f	floating-point	Defaults to exactly 6 decimal places unless given precision
%.2f	floating-point	Exactly 2 decimal places
%n	(none)	Platform-correct line separator -- preferred over a literal <code>\n</code>

POSITIONAL MATCHING

The 1st placeholder pairs with the 1st argument after the format string, the 2nd with the 2nd, and so on -- pure left-to-right order, nothing name-based.

TYPE-CHECK TIMING

printf accepts a generic `Object...` argument list, so the compiler cannot check placeholder/argument type agreement. A mismatch (e.g. `%d` given a `String`) compiles cleanly and throws `IllegalFormatConversionException` only when the line actually executes.

SOURCE [java.util.Formatter \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Formatter.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Formatter.html>

SEE ALSO [System.out.print / println / printf](#)

Scanner SYNTAX

Reads input by breaking it into tokens using a delimiter pattern that defaults to whitespace -- requires `import java.util.Scanner`, unlike `java.lang` classes.

MECHANISM

`next*/nextInt/nextDouble/next()` read a token only up to the next delimiter (whitespace, including the newline from Enter) and leave that delimiter unconsumed in the buffer.

EXAMPLE

```
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
int age = sc.nextInt();
double price = sc.nextDouble();
```

SOURCE [java.util.Scanner \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Scanner.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Scanner.html>

SEE ALSO [Scanner nextLine buffer issue](#) · [import](#)

Scanner nextLine buffer issue CONCEPT

Any `next*` method (`nextInt`, `nextDouble`, `next`) leaves the trailing newline unconsumed; if `nextLine()` is called right after, it immediately grabs that leftover newline and returns an empty string, skipping the real input entirely.

MECHANISM

`nextInt()` reads only digit characters and stops at the newline -- it never consumes it. The very next thing in the input queue is that leftover newline, so a following `nextLine()` call finds it immediately and returns "" without ever reaching the real text typed afterward.

FIX

Insert exactly one throwaway `sc.nextLine();` (its return value discarded) right after the `next*/nextInt/nextDouble` call, before the real `nextLine()` call that should capture input. Adding a SECOND unnecessary throwaway call consumes actual input instead and can crash with `NoSuchElementException` once the input runs out.

EXAMPLE

```
int age = sc.nextInt();
sc.nextLine();           // discard the leftover newline
String name = sc.nextLine(); // now captures the real input
```

GENERAL RULE

`nextLine()` called on its own, back-to-back, has no leftover problem -- it is only ever caused by a DIFFERENT `next*` method running immediately before it.

SOURCE [java.util.Scanner \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Scanner.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Scanner.html>

SEE ALSO [Scanner](#)

TOPIC AREA

8 entries

Dates & Times

AT A GLANCE

The `java.time` types differ by exactly what they carry — a date, a time, both, both plus a zone, or a machine-timeline instant — and the right one is the smallest that holds what the problem needs. Every value is immutable, so arithmetic returns a new object and must be assigned. Amounts split the same way: `Duration` measures exact elapsed time and `Period` measures calendar time, and across a daylight-saving change they deliberately give different answers.

java.time types CONCEPT

Five types that differ only in what they carry; choosing correctly means picking the smallest one that holds what the problem actually needs, because every extra thing carried is another thing that can go wrong.

THE FIVE TYPES

TYPE	CARRIES	DOES NOT CARRY	EXAMPLE VALUE
LocalDate	a calendar date	any time or zone	2026-01-31
LocalTime	a time of day	any date or zone	09:05:30
LocalDateTime	a date and a time	a zone — so it is not a real moment	2026-01-31T09:05
ZonedDateTime	a date, a time, and a zone's rules	nothing — a real moment, subject to DST	2026-03-08T01:30-05:00[America/New_York]
Instant	a point on the machine timeline (UTC)	human calendar context	2026-07-17T14:30:00Z

MECHANISM

The prefix `Local` means no zone is attached. A `LocalDateTime` of 09:05 is a reading any clock anywhere could show; it is not a moment in the world until a zone is attached. Attaching one produces a `ZonedDateTime`, and that is the precise point at which daylight-saving rules start applying to arithmetic.

USE WHEN

A birthday is a `LocalDate` -- it does not shift when you fly. A meeting across cities is a `ZonedDateTime`. A log timestamp is an `Instant`. Reaching for a bigger type than the problem needs is what imports zone bugs into code that never needed a zone.

NOTE

All five live in `java.time` and all are immutable.

SOURCE [java.time \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html>

SEE ALSO [creating date-time values](#) · [date-time immutability](#) · [ZonedDateTime & DST](#)

creating date-time values SYNTAX

Built through static factory methods -- `of()`, `now()`, and `parse()` -- never with `new`, because these classes have no public constructors.

MECHANISM

The `java.time` classes hide their constructors, so `new LocalDate()` does not compile. Factories are the only door in. Months are 1-based here: 1 is January, unlike the 0-based month of the legacy `java.util.Calendar` API.

THE THREE WAYS

<code>of(...)</code>	build from explicit parts -- <code>LocalDate.of(2026, 1, 31)</code>
<code>now()</code>	read the current value from the system clock -- <code>LocalDate.now()</code>
<code>parse(text)</code>	build from ISO text -- <code>LocalDate.parse("2026-01-31")</code>

VERIFIED BEHAVIOR (JDK 21)

An impossible date throws instead of rolling over: `LocalDate.of(2026, 2, 30)` raises `DateTimeException`. It does not silently become March 2 the way a lenient date API would.

EXAMPLE

```
LocalDate d = LocalDate.of(2026, 1, 31);    // 2026-01-31
LocalDate t = LocalDate.now();
LocalDate p = LocalDate.parse("2026-01-31");
// LocalDate x = new LocalDate();           // does not compile
```

SOURCE [java.time \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html>

SEE ALSO [java.time types](#) · [date-time immutability](#)

date-time immutability CONCEPT

Every plus/minus/with call returns a NEW value and leaves the original untouched, so a call whose result is not assigned does nothing at all.

MECHANISM

A date-time object can never change after it is created. `plusDays(1)` computes a new object and hands it back; the variable it was called on still holds the old value. This is the identical rule `String` follows, and for the identical reason -- a value that can be shared safely can only ever hand back a new one.

VERIFIED BEHAVIOR (JDK 21)

Month-end arithmetic clamps rather than overflowing: `2026-01-31 plusMonths(1)` gives `2026-02-28`, the last valid day of the target month, with no exception raised.

EXAMPLE

```
LocalDate d = LocalDate.of(2026, 1, 31);
d.plusDays(1);           // returns 2026-02-01 -- thrown away
System.out.println(d);  // 2026-01-31 -- unchanged

d = d.plusDays(1);      // the working form
```

SOURCE [java.time \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html>

SEE ALSO [String immutability](#) · [java.time types](#) · [creating date-time values](#)

Period vs Duration CONCEPT

Duration measures exact elapsed time and is blind to calendars; Period measures calendar time and is blind to exact seconds -- and the choice only reveals itself where the two disagree.

THE TWO AMOUNTS

TYPE	MEASURES IN	BLIND TO	TOSTRING FORM
Duration	hours, minutes, seconds, nanos	calendars -- a day is always exactly 24h to it	PT2H · PT1H30M
Period	years, months, days	exact seconds -- a day is whatever the calendar says	P1Y2M3D · P1M1D

MECHANISM

A daylight-saving change is the place the two part company. On a spring-forward date the calendar day is only 23 real hours long. Adding one calendar day and adding 24 exact hours therefore land on different clock times. Neither is wrong -- they answer different questions, and the directional consequence follows from which frame is held fixed: Period holds the clock reading fixed and lets elapsed time vary; Duration holds elapsed time fixed and lets the clock reading vary.

VERIFIED BEHAVIOR (JDK 21)

```
ZoneId ny = ZoneId.of("America/New_York");
ZonedDateTime start = ZonedDateTime.of(
    LocalDateTime.of(2026, 3, 7, 12, 0), ny); // 12:00 -05:00, day before spring-
forward

start.plusDays(1); // 2026-03-08T12:00-04:00 <- same clock time
start.plus(Duration.ofHours(24)); // 2026-03-08T13:00-04:00 <- an hour later on the
clock

// real hours between start and plusDays(1): 23
```

USE WHEN

Ask which frame the answer must honour. "Same time tomorrow" is a calendar promise -- Period or plusDays. "24 hours from now" is a stopwatch promise -- Duration. Choosing by unit size alone (days feel big, hours feel small) is what selects the wrong one.

NOTE

The ofX factories are static, so chaining them does not accumulate: Period.ofYears(1).ofMonths(2) evaluates to P2M -- only the last call survives. Build multi-field amounts with Period.of(1, 2, 0).

SOURCE [java.time \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html>

SEE ALSO [Period.between\(\)](#) · [ZonedDateTime & DST](#) · [java.time types](#)

Period.between() CONCEPT

Reports the gap between two dates by removing complete calendar months first, then counting the days left over from wherever that landed -- not by dividing a total day count.

MECHANISM

It advances from the start date by whole calendar months for as long as it can without passing the end date, then counts the remaining days from that landing point. Because months differ in length, the day part depends entirely on where the month-stepping stopped -- which is why the result cannot be reproduced by dividing a total number of days.

VERIFIED BEHAVIOR (JDK 21)

```
Period p = Period.between(
    LocalDate.of(2024, 1, 31),
    LocalDate.of(2024, 3, 1));
// P1M1D -- one month, one day

// why: Jan 31 + 1 month clamps to Feb 29 (2024 is a leap year),
// and Feb 29 -> Mar 1 is one further day.
```

USE WHEN

Reporting an age or a gap in human calendar terms. For a plain exact count instead, use ChronoUnit.DAYS.between(), which returns a single total and never splits into months.

NOTE

Argument order matters: a start date later than the end date yields a negative Period.

SOURCE [java.time \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html>

SEE ALSO [Period vs Duration](#) · [date-time immutability](#)

ZonedDateTime & DST CONCEPT

A ZonedDateTime carries a region's daylight-saving rulebook, so zoned arithmetic adjusts the UTC offset automatically and some calendar days are not 24 hours long.

MECHANISM

A zone is a rulebook, not a fixed offset. America/New_York means -05:00 in winter and -04:00 in summer, and the rulebook states when it flips. On the spring-forward date the 2:00 hour does not exist at all, making that calendar day 23 real hours; on the fall-back date one hour occurs twice, making it 25.

SPRING-FORWARD, 2026-03-07 12:00 NEW YORK (VERIFIED JDK 21)

12:00 -05:00 START	→	plusDays(1) CALENDAR +1 DAY	→	12:00 -04:00 23 REAL HOURS LATER
---	---	--	---	---

USE WHEN

Any time a real moment must survive a zone or a DST boundary. A LocalDateTime has no rulebook attached and therefore cannot be affected by any of this -- which is a positive reason to prefer it when zones genuinely do not matter.

NOTE

Constructing a time inside the skipped hour does not throw; it is shifted forward to a real time instead.

SOURCE [java.time \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html>

SEE ALSO [zone conversion & the date shift](#) · [Period vs Duration](#) · [java.time types](#)

zone conversion & the date shift CONCEPT

Viewing one moment from another zone can move the wall clock across midnight, changing the reported calendar date -- the real danger of routing a calendar-only concept through a `ZonedDateTime`.

MECHANISM

`withZoneSameInstant` keeps the moment identical and re-reads it on a different region's clock. If that clock has already passed midnight, the same instant reports a different calendar date. Nothing moved in time; only the viewpoint changed.

VERIFIED BEHAVIOR (JDK 21)

```
ZonedDateTime ny = ZonedDateTime.of(
    LocalDateTime.of(2000, 3, 15, 20, 0),
    ZoneId.of("America/New_York"));    // 2000-03-15T20:00-05:00

ny.withZoneSameInstant(ZoneId.of("Asia/Tokyo"));
// 2000-03-16T10:00+09:00 -- same instant, NEXT DAY's date
```

USE WHEN

Pulling to `LocalDate()` off a converted `ZonedDateTime` can hand back a date the user would not recognise as theirs. If the concept is fundamentally a date -- a birthday, a due date -- keep it a `LocalDate` and never route it through a zone at all.

NOTE

Contrast with `withZoneSameLocal`, which keeps the clock reading and changes the moment instead -- the opposite trade.

SOURCE [java.time \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/package-summary.html>

SEE ALSO [ZonedDateTime & DST](#) · [java.time types](#)

DateTimeFormatter patterns SYNTAX

Formats a date-time into text or parses text back, driven by pattern letters where CASE selects the field and the letter COUNT selects the width or verbosity.

MECHANISM

Upper and lower case are different fields entirely, not styles of one field: MM is month and mm is minute. The number of letters then chooses how that field is rendered. This is why a wrong-case pattern still compiles, still runs, and simply prints the wrong value -- there is no error to catch it.

CASE DECIDES THE FIELD (VERIFIED JDK 21, ON 2026-01-31T13:05)

PATTERN	MEANS	OUTPUT
MM	month	01
mm	minute	05
HH	hour, 24-hour clock	13
hh	hour, 12-hour clock	01
a	AM/PM marker	PM
dd	day of month	31
yyyy	year	2026

LETTER COUNT DECIDES VERBOSITY

PATTERN	OUTPUT
M	1
MM	01
MMM	Jan
MMMM	January

EXAMPLE

```
DateTimeFormatter f = DateTimeFormatter.ofPattern("MM/dd/yyyy");
f.format(LocalDate.of(2026, 1, 31));           // 01/31/2026
LocalDate.parse("01/31/2026", f);             // 2026-01-31

DateTimeFormatter.ISO_LOCAL_DATE.format(d); // 2026-01-31
```

USE WHEN

Reach for a predefined constant such as `ISO_LOCAL_DATE` when standard ISO text is what is wanted; write `ofPattern` only for a layout the constants do not already cover.

SOURCE [DateTimeFormatter \(API, SE 25\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/format/DateTimeFormatter.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/format/DateTimeFormatter.html>

SEE ALSO [java.time types](#) · [creating date-time values](#)

Index

Every term, alphabetical, with its page.

<code>==</code> vs <code>.equals()</code>	12	operator precedence	14
binary numeric promotion	15	Period vs Duration	21
bytecode	4	Period.between()	22
constant expression narrowing	8	primitive types	6
creating date-time values	20	printf format specifiers	17
date-time immutability	20	Scanner	18
DateTimeFormatter patterns	23	Scanner nextLine buffer issue	18
double	7	String	10
How Java runs	3	String immutability	10
import	5	string pool	11
int	7	StringBuilder	12
integer division & modulus	14	System.out.print / println / printf ..	17
java	4	text block whitespace	13
java.time types	19	text blocks	13
javac	3	var	7
JDK vs JRE vs JVM	4	variable declaration	7
literal defaults & suffixes	8	widening & narrowing conversion	15
main method	5	zone conversion & the date shift	23
Math class	15	ZonedDateTime & DST	22
new String()	11		